

CSE344 SYSTEMS PROGRAMMING
FINAL REPORT

Burak Demirkaya

210104004274

14/06/2024

1. Introduction

This homework provides an overview of a multithreaded Pide Shop simulation implemented in C. The simulation involves multiple entities such as orders, cooks, and delivery personnel, each represented as a struct in the code.

2. Design Decisions

The Order struct represents an individual order placed by a customer. It includes details such as the unique order ID, the location of the customer, the status of the order, the timestamp when the order was placed, the client socket associated with the order, and a flag indicating whether the order has been picked up.

The Cook struct represents a cook in the Pide Shop. It includes a unique ID, a thread representing the cook's work, a flag indicating whether the cook is busy, and a count of the total number of orders the cook has prepared.

The DeliveryPersonnel struct represents a delivery person. It includes a unique ID, a flag indicating whether the delivery person is busy, the delivery person's location, a thread representing the delivery person's work, an array of orders to be delivered, a count of the number of orders to be delivered, the delivery speed, and a count of the total number of orders the delivery person has delivered.

```
typedef struct {
    int order_id; // Unique identifier for the order
    int x, y; // Location of the customer
    int status; // 0: Received, 1: Preparing, 2: Cooked, 3:Delivered
    time_t timestamp; // Time of the order
    int client_socket; // Client socket
    int pickedUp; // 0: Not picked up, 1: Picked up
} Order;

typedef struct {
    int id; // Unique identifier for the cook
    pthread_t thread; // Thread for the cook
    int busy; // 0: Free, 1: Busy
    int totalOrdersCooked; // Total number of orders cooked
} Cook;

typedef struct {
    int id; // Unique identifier for the delivery personnel
    int busy; // 0: Free, 1: Busy
    int x, y; // Location of the delivery personnel
    pthread_t thread; // Thread for the delivery personnel
    Order orders[MAX_DELIVERY_CAPACITY]; // Orders to be delivered
    int orderCount; // Number of orders to be delivered
    int deliverySpeed; // Speed of the delivery personnel
    int totalOrdersDelivered; // Total number of orders delivered
} DeliveryPersonnel;
```

```

Order orders[MAX_ORDER_COUNT]; // Queue for the orders max 500 orders
int orderCount = 0;
Order readyOrders[MAX_ORDER_COUNT]; // Queue for the orders ready for delivery
int readyOrderCount = 0;

Cook *cooks; // Cooks
DeliveryPersonnel *deliveryPersonnel; // Delivery personnels

int serverRunning = 1;
int townX, townY; // Location of the shop
pthread_mutex_t orderLock; // Mutex for the orders
pthread_mutex_t deliveryLock; // Mutex for the delivery personnel
pthread_mutex_t readyOrderLock; // Mutex for the ready orders
pthread_cond_t orderTaken; // Condition variable for the order
int ordersLeft = 0; // Number of orders left

int cookPoolSize; // Number of cooks
int deliveryPoolSize; // Number of delivery personnel

sem_t ovenCapacity; // Semaphore for the oven capacity
sem_t ovenApparatus; // Semaphore for the oven apparatus
pthread_mutex_t ovenLock; // Mutex for the oven

pthread_cond_t orderReadyForDelivery; // Condition variable for the delivery personnel
pthread_cond_t everyOrderDelivered; // Condition variable for the delivery personnel

int server_socket; // Server socket
int totalOrders = -10; // Total number of orders
int portNumber; // Port number
FILE *logFile; // Log file
int clientPID; // Client PID

```

Global

variables and mutexes I have defined in my program.

Server Side:

- **Handle_signal**

This function is designed to handle the SIGINT signal. When this signal is received, the function prints a message to the console, writes the total number of orders to the log file, sets the serverRunning flag to 0 to indicate that the server is no longer running, and then closes the server socket.

```

void handle_signal(int signal){
    if(signal == SIGINT){
        printf(".. Upps quitting... writing to log file\n");
        fprintf(logFile, "Total number of orders: %d\n", totalOrders);
        serverRunning = 0;
        close(server_socket);
    }
}

```

- **createSocketServer**

The createSocketServer function is responsible for setting up a server socket. It first creates socket then sets up a sockaddr_in structure with the server's address details: it uses the Internet address family (AF_INET), the provided port number, and accepts connections to any IP address the machine happens to have. It then binds the socket to the specified address and port number. Finally, it sets the socket to listen for incoming connections, with a backlog limit of 10 clients.

```
void createSocketServer(int port){
    server_socket = socket(AF_INET, SOCK_STREAM, 0); // Create a socket
    if(server_socket == -1){
        perror("socket");
        exit(EXIT_FAILURE);
    }

    struct sockaddr_in server_address;
    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(portNumber);
    server_address.sin_addr.s_addr = INADDR_ANY;

    if(bind(server_socket, (struct sockaddr*)&server_address, sizeof(server_address)) == -1){
        perror("bind");
        exit(EXIT_FAILURE);
    }

    if(listen(server_socket, 10) == -1){ // 10 is the maximum number of clients that can be wa
        perror("listen");
        exit(EXIT_FAILURE);
    }
}
```

- **managerThread**

The managerThread function is the main loop of the server application. It continuously checks for incoming connections and processes orders from clients. When a client connects, the server reads the order and stores it in an Order structure. The order's status is set to 0 (received), a timestamp is added, and the client's socket is stored for later communication. The server then sends a response back to the client confirming the receipt of the order.

If it's the first order, the server prints a message indicating the total number of orders it will serve. The order is then added to the orders array and the orderTaken condition is signaled to wake up any waiting cook threads.

The server also checks if all orders have been delivered. If so, it prints the best cook and delivery personnel, resets the order count and total orders, and continues to the next iteration.

The function also handles order cancellation. If the client sends a "Cancel" message, the server will parse the order ID from the message and print a cancellation message. (Does not work correctly)

The server uses the select function to wait for activity on the server socket with a timeout. If there's no activity within the timeout period, the server continues to the next iteration of the loop. If there's an error with select, the server prints an error message and exits.

The function uses mutex locks (orderLock) to protect shared resources (orders, orderCount, ordersLeft, etc.) from concurrent access by multiple threads. It uses condition variables (everyOrderDelivered, orderTaken) to synchronize the execution of threads based on the state of these shared resources.

```
void* managerThread(void* arg){ // Manager thread that will handle the orders
    printf("PideShop active waiting for connection...\n");

    int firstOrder = 1;
    char response[1024];
    fd_set readfds;
    struct timeval timeout; // Timeout for the select function

    while(serverRunning){
        if(orderCount == totalOrders){
            if(pthread_mutex_lock(&orderLock) != 0){
                perror("pthread_mutex_lock");
                exit(EXIT_FAILURE);
            }
            while(ordersLeft > 0){
                if(pthread_cond_wait(&everyOrderDelivered, &orderLock) != 0){
                    perror("pthread_cond_wait");
                    exit(EXIT_FAILURE);
                }
                if(!serverRunning){
                    break;
                }
            }
            if(ordersLeft == 0){
                printf("Done serving client PID %d\n", clientPID);
                printBestCookAndDeliveryPersonnel();
                orderCount = 0;
                ordersLeft = 0;
                totalOrders = -10;
                firstOrder = 1;
                if(pthread_mutex_unlock(&orderLock) != 0){
                    perror("pthread_mutex_unlock");
                    exit(EXIT_FAILURE);
                }
            }
        }
        FD_ZERO(&readfds);
        FD_SET(server_socket, &readfds);
        timeout.tv_sec = 1;
        timeout.tv_usec = 0;

        int activity = select(server_socket + 1, &readfds, NULL, NULL, &timeout); // Wait for the client
        if(activity == -1){
            perror("select");
            exit(EXIT_FAILURE);
        } else if(activity == 0){
            continue;
        }

        if(FD_ISSET(server_socket, &readfds)){
            int client_socket = accept(server_socket, NULL, NULL); // Accept the client
            if(client_socket == -1){
                if(!serverRunning){
                    break;
                }
                perror("accept");
                continue;
            }

            char buffer[1024] = {0};
            int bytesRead = read(client_socket, buffer, 1024); // Read the order from the client
            if(bytesRead == -1){
                perror("read");
                exit(EXIT_FAILURE);
            }

            if(strncmp(buffer, "Cancel", 6) == 0){
                int order_id;
                if(sscanf(buffer, "Cancel %d", &order_id) != 1){
                    perror("sscanf");
                    exit(EXIT_FAILURE);
                }
                printf("Order %d is cancelled\n", order_id);
            }
        }
    }
}
```

```

int q,p; // Team sizes

Order order = {0};
if (scanf("Order %d: Location (%d, %d) %d Team Size (%d, %d) PID %d", &order.order_id, &order.x, &order.y, &totalOrders, &p, &q, &client))
    perror("scanf");
    exit(EXIT_FAILURE);
}

teamX = p / 2;
teamY = q / 2;

order.status = 0;
order.timestamp = time(NULL);
order.client_socket = client_socket;
order.pickedUp = 0;

sprintf(response, "Order %d received\n", order.order_id);
if (send(client_socket, response, strlen(response), 0) == -1){
    perror("send manager thread");
    exit(EXIT_FAILURE);
}

if (firstOrder){
    printf("%d new customers... Serving\n", totalOrders);
    firstOrder = 0;
}

if (pthread_mutex_lock(&orderLock) != 0){
    perror("pthread_mutex_lock");
    exit(EXIT_FAILURE);
}

if (orderCount < MAX_ORDER_COUNT){
    orders[orderCount++] = order;
    ordersLeft++;
    if (pthread_cond_broadcast(&orderTaken) != 0){
        perror("pthread_cond_signal");
        exit(EXIT_FAILURE);
    }
} else{
    printf("Order queue is full\n");
}

if (pthread_mutex_unlock(&orderLock) != 0){
    perror("pthread_mutex_unlock");
    exit(EXIT_FAILURE);
}

}

return NULL;
}

```

After every order has been delivered manager thread prints the best cook and delivery personnel depending on their performance.

```

void printBestCookAndDeliveryPersonnel(){
    int numberOfCooked = 0;
    int bestCookId = 0;
    for(int i = 0; i < cookPoolSize; i++){
        if (cooks[i].totalOrdersCooked > numberOfCooked){
            numberOfCooked = cooks[i].totalOrdersCooked;
            bestCookId = i;
        }
    }

    int numberOfDelivered = 0;
    int bestDeliveryPersonnelId = 0;
    for(int i = 0; i < deliveryPoolSize; i++){
        if (deliveryPersonnel[i].totalOrdersDelivered > numberOfDelivered){
            numberOfDelivered = deliveryPersonnel[i].totalOrdersDelivered;
            bestDeliveryPersonnelId = i;
        }
    }

    printf("Thanks Cook %d (%d orders) and Moto %d (%d deliveries)\n", bestCookId, cooks[bestCookId].totalOrdersCooked, bestDeliveryPersonnelId, deliveryPer
}

```

- **cookThread**

This thread operates in a loop that continues as long as the serverRunning variable is true. The function uses several synchronization primitives to ensure that multiple cooks can operate concurrently without interfering with each other:

A mutex (orderLock) is used to ensure that only one cook can access the orders at a time.

A condition variable (orderTaken) is used to allow a cook to wait until there is an order to prepare.

Two semaphores (ovenApparatus and ovenCapacity) are used to manage access to the oven.

The cook checks if there are any orders to prepare. If there are no orders, the cook waits on the orderTaken condition variable. Once an order is available, the cook prepares the order. This involves performing some matrix operations and then waiting for half the preparation time.

The cook then places the order in the oven. This involves waiting on the ovenApparatus and ovenCapacity semaphores to ensure that the oven is available and has capacity. Once the order is cooked, the cook places the order in the readyOrders array and signals the orderReadyForDelivery condition variable to indicate that an order is ready for delivery. Finally, the cook sends a message to the client indicating that the order is cooked and then sets itself to be not busy.

```
void* cookThread(void* arg) { // Cook thread that will cook the orders
    Cook *cook = (Cook*)arg;
    while(serverRunning){
        if(pthread_mutex_lock(&orderLock) != 0){
            perror("pthread_mutex_lock");
            // exit(EXIT_FAILURE);
            continue;
        }

        while(orderCount == 0 && serverRunning){
            if(pthread_cond_wait(&orderTaken, &orderLock) != 0){
                perror("pthread_cond_wait");
                pthread_mutex_unlock(&orderLock);
                // exit(EXIT_FAILURE);
                continue;
            }
        }

        if(!serverRunning){
            printf("Cook %d is quitting\n", cook->id);
            pthread_cond_broadcast(&orderReadyForDelivery);
            pthread_mutex_unlock(&orderLock);
            break;
        }

        char response[1024];
        int orderIndex = -1;
        for(int i=0; i<orderCount; i++){
            if(orders[i].status == 0 && !cook->busy){ // If the order is received
                orders[i].status = 1; // Set the status to preparing
                orderIndex = i;
                break;
            }
        }

        if(orderIndex == -1){ // If no order is found
            pthread_mutex_unlock(&orderLock);
            // exit(EXIT_FAILURE);
            continue;
        }

        // write to log file
        if(fprintf(logFile, "Order %d prepared in %f second\n", orders[orderIndex].order_id, time) < 0){
            perror("fprintf");
            sem_post(&ovenApparatus);
            sem_post(&ovenCapacity);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        memset(response, 0, sizeof(response));
        sprintf(response, "Order %d prepared by cook %d\n", orders[orderIndex].order_id, cook->id);
        if(send(orders[orderIndex].client_socket, response, strlen(response), 0) == -1){
            perror("send cookThread prepare");
            sem_post(&ovenApparatus);
            sem_post(&ovenCapacity);
            // exit(EXIT_FAILURE);
            continue;
        }

        if(pthread_mutex_unlock(&ovenLock) != 0){
            perror("pthread_mutex_unlock");
            sem_post(&ovenApparatus);
            sem_post(&ovenCapacity);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(sem_post(&ovenApparatus) == -1){ // Release the oven apparatus
            perror("sem_post");
            sem_post(&ovenCapacity);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(pthread_mutex_unlock(&orderLock) != 0){
            perror("pthread_mutex_unlock");
            continue;
        }

        struct timeval start, end;
        gettimeofday(&start, NULL); // Start the timer
        initializeMatrix();
        transposeMatrix();
        multiplyMatrix();
        inverseMatrix();
        gettimeofday(&end, NULL); // End the timer
        double time = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;

        if(sem_wait(&ovenApparatus) == -1){ // Wait for the oven apparatus
            perror("sem_wait");
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(sem_wait(&ovenCapacity) == -1){ // Wait for the oven capacity
            perror("sem_wait");
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(pthread_mutex_lock(&ovenLock) != 0){
            perror("pthread_mutex_lock");
            sem_post(&ovenApparatus);
            sem_post(&ovenCapacity);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        sleep((int)(time/2.0)); // Wait for the order to be cooked
        orders[orderIndex].status = 2; // Set the status to cooked

        if(sem_wait(&ovenApparatus) == -1){ // Wait for the oven apparatus
            perror("sem_wait");
            sem_post(&ovenCapacity);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(sem_post(&ovenCapacity) == -1){
            perror("sem_post");
            sem_post(&ovenApparatus);
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(sem_post(&ovenApparatus) == -1){
            perror("sem_post");
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }

        if(pthread_mutex_lock(&readyOrderLock) != 0){
            perror("pthread_mutex_lock");
            cook->busy = 0;
            // exit(EXIT_FAILURE);
            continue;
        }
    }
}
```

```

    if (readyOrderCount < MAX_ORDER_COUNT && !orders[orderIndex].pickedUp && orders[orderIndex].status == 2) {
        orders[orderIndex].pickedUp = 1; // Mark as picked up to avoid duplicates
        readyOrders[readyOrderCount++] = orders[orderIndex];
        // printf("Order %d is ready for delivery\n", orders[i].order_id);
        if (pthread_cond_broadcast(&orderReadyForDelivery) != 0) {
            perror("pthread_cond_broadcast");
            pthread_mutex_unlock(&readyOrderLock);
            cook->busy = 0;
            continue;
        }
    } else {
        printf("Ready order queue is full or order already picked up\n");
    }

    if (fprintf(logFile, "Order %d cooked\n", orders[orderIndex].order_id) < 0) {
        perror("fprintf");
        cook->busy = 0;
        continue;
    }

    sprintf(response, "Order %d is cooked by cook %d\n", orders[orderIndex].order_id, cook->id);
    if (send(orders[orderIndex].client_socket, response, strlen(response), 0) == -1) {
        perror("send cookThread cook");
        cook->busy = 0;
        continue;
    }

    if (pthread_mutex_unlock(&readyOrderLock) != 0) {
        perror("pthread_mutex_unlock");
        cook->busy = 0;
        continue;
    }

    cook->busy = 0; // Set the cook to free
    cook->totalOrdersCooked++; // Increment the total orders cooked by the cook
}
return NULL;
}

```

```

struct timeval start, end; // Timer for preparing the order
gettimeofday(&start, NULL); // Start the timer
initializeMatrix();
transposeMatrix();
multiplyMatrix();
inverseMatrix();
gettimeofday(&end, NULL); // End the timer
double time = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;

```

This is how I handle the prepare time while calculating the pseudo-inverse of 30x40 matrix with complex numbers.

- **deliveryThread**

The thread operates in a loop that continues as long as the serverRunning variable is true. The function uses several synchronization primitives to ensure that multiple delivery personnel can operate concurrently without interfering with each other:

A mutex (deliveryLock) is used to ensure that only one delivery person can access the ready orders at a time.

A condition variable (orderReadyForDelivery) is used to allow a delivery person to wait until there are at least three orders ready for delivery.

The delivery person checks if there are at least three orders ready for delivery. If there are not, the delivery person waits on the orderReadyForDelivery condition variable.

Once there are enough orders, the delivery person picks up to three orders from the readyOrders array and marks them as picked up. The delivery person then delivers each order one by one. This involves calculating the delivery time, simulating the delivery by sleeping for the delivery time, updating the order status and timestamp, and sending a message to the client indicating that the

order has been delivered. After all orders have been delivered, the delivery person returns to the shop and updates their delivery statistics. If all orders have been delivered, the delivery person signals the everyOrderDelivered condition variable to indicate that all orders have been delivered.

```
void* deliveryThread(void* arg) {
    DeliveryPersonnel *deliveryPersonnel = (DeliveryPersonnel*)arg;

    deliveryPersonnel->x = townX; // Set the initial x location of the delivery personnel
    deliveryPersonnel->y = townY; // Set the initial y location of the delivery personnel

    while (serverRunning) {
        if (pthread_mutex_lock(&deliveryLock) != 0) {
            perror("pthread_mutex_lock");
            exit(EXIT_FAILURE);
        }

        while (readyOrderCount < 3 && serverRunning) {
            if (pthread_cond_wait(&orderReadyForDelivery, &deliveryLock) != 0) {
                perror("pthread_cond_wait");
                pthread_mutex_unlock(&orderLock);
                exit(EXIT_FAILURE);
            }
        }

        if (!serverRunning) {
            printf("Delivery %d is quitting\n", deliveryPersonnel->id);
            pthread_mutex_unlock(&deliveryLock);
            break;
        }

        int ordersToDeliver = (readyOrderCount >= 3) ? 3 : readyOrderCount;
        for (int i = 0; i < ordersToDeliver; i++) {
            deliveryPersonnel->orders[i] = readyOrders[i];
            deliveryPersonnel->orders[i].pickedUp = 1;
        }

        // Shift remaining orders in the queue
        for (int i = ordersToDeliver; i < readyOrderCount; ++i) {
            readyOrders[i - ordersToDeliver] = readyOrders[i];
        }
        readyOrderCount -= ordersToDeliver;

        deliveryPersonnel->orderCount = ordersToDeliver;
        deliveryPersonnel->busy = 1;

        if (pthread_mutex_unlock(&deliveryLock) != 0) {
            perror("pthread_mutex_unlock");
            exit(EXIT_FAILURE);
        }

        for (int i = 0; i < deliveryPersonnel->orderCount; ++i) {
            if (!serverRunning) {
                if (pthread_cond_signal(&everyOrderDelivered) != 0) { // Ensure manager thread is not stuck
                    perror("pthread_cond_broadcast");
                    exit(EXIT_FAILURE);
                }
            }
            close(deliveryPersonnel->orders[i].client_socket);
            return NULL;
        }
        Order order = deliveryPersonnel->orders[i];

        // Calculate delivery time
        int deliveryTime = calculateDeliveryTime(order.x, order.y, deliveryPersonnel->deliverySpeed, *deliveryPersonnel);

        // Simulate delivery
        sleep(deliveryTime);

        // Update order status
        order.status = 3; // Delivered
        order.timestamp = time(NULL);

        deliveryPersonnel->x = order.x;
        deliveryPersonnel->y = order.y;

        // printf("Order %d delivered to (%d, %d) by delivery %d\n", order.order_id, order.x, order.y, deliveryPersonnel->id);

        if ((printf(logfile, "Order %d delivered to (%d, %d) by delivery %d\n", order.order_id, order.x, order.y, deliveryPersonnel->id) < 0)) {
            perror("fprintf");
            exit(EXIT_FAILURE);
        }
    }

    char response[1024];
    sprintf(response, "Order %d delivered by delivery %d\n", order.order_id, deliveryPersonnel->id);
    if (send(order.client_socket, response, strlen(response), 0) == -1) {
        perror("send deliveryThread");
        exit(EXIT_FAILURE);
    }
    ordersLeft--;

    close(order.client_socket);

    // Update order in the global orders array
    for (int j = 0; j < orderCount; j++) {
        if (orders[j].order_id == order.order_id) {
            orders[j] = order;
            break;
        }
    }

    // Simulate return to the shop
    sleep(1);
    deliveryPersonnel->x = townX;
    deliveryPersonnel->y = townY;

    // Update delivery statistics
    deliveryPersonnel->totalOrdersDelivered += deliveryPersonnel->orderCount;

    // Reset the order count for the next round of deliveries
    deliveryPersonnel->orderCount = 0;
    deliveryPersonnel->busy = 0;
    memset(deliveryPersonnel->orders, 0, sizeof(deliveryPersonnel->orders));

    if (ordersLeft == 0) {
        printf("All orders delivered\n");
        if (pthread_cond_signal(&everyOrderDelivered) != 0) {
            perror("pthread_cond_broadcast");
            exit(EXIT_FAILURE);
        }
    }

    return NULL;
}

int calculateDeliveryTime(int x, int y, int deliverySpeed, DeliveryPersonnel deliveryPersonnel) {
    int distance = (int)sqrt(pow(deliveryPersonnel.x - x, 2) + pow(deliveryPersonnel.y - y, 2));
    int time = distance / deliverySpeed;
    // printf("Delivery (%d,%d) time: %d\n", x, y, time);
    return time;
}
```

This is how delivery personnel calculate the time depending on his/her location and the order location.

```
pthread_t manager_Thread;
if(pthread_create(&manager_Thread, NULL, managerThread, NULL) != 0){
    perror("pthread_create");
    exit(EXIT_FAILURE);
}

cooks = (Cook*)malloc(cookPoolSize * sizeof(Cook));
for(int i = 0; i < cookPoolSize; i++){
    cooks[i].id = i;
    cooks[i].busy = 0;
    cooks[i].totalOrdersCooked = 0;
    pthread_create(&cooks[i].thread, NULL, cookThread, &cooks[i]);
}
deliveryPersonnel = (DeliveryPersonnel*)malloc(deliveryPoolSize * sizeof(DeliveryPersonnel));
for(int i = 0; i < deliveryPoolSize; i++){
    deliveryPersonnel[i].id = i;
    deliveryPersonnel[i].busy = 0;
    deliveryPersonnel[i].orderCount = 0;
    deliveryPersonnel[i].totalOrdersDelivered = 0;
    deliveryPersonnel[i].deliverySpeed = deliverySpeed;
    pthread_create(&deliveryPersonnel[i].thread, NULL, deliveryThread, &deliveryPersonnel[i]);
}
```

Main thread of server side simply initializes each mutex, condition variables and semaphores and creates corresponding threads for manager, cooks and delivery personnel.

Client Side

- **Handle_signal**

```
void handle_signal(int signal){
    if(signal == SIGINT){
        printf("signal... cancelling orders... editing log\n");

        for(int i=0; i<numberOfClients; i++){
            char buffer[1024] = {0};
            sprintf(buffer, "Cancel %d", i+1);
            send(client_sockets[i], buffer, strlen(buffer), 0);
        }

        for(int i=0; i<numberOfClients; i++){
            close(client_sockets[i]);
        }
        // exit(EXIT_SUCCESS);
    }
}
```

Handle signal sends cancel request to each of the client sockets. (Does not work correctly)

- **Handle_client**

```
void send_order(int sock, int order_id, int x, int y, int townX, int townY) {
    char buffer[1024] = {0};
    sprintf(buffer, "Order %d: Location (%d, %d) %d Town Size (%d, %d) PID %d", order_id, x, y, numberOfClients, townX, townY, getpid());
    send(sock, buffer, strlen(buffer), 0);
    // printf("Order %d sent", order_id);
}

void* handle_client(void* arg){
    int sock = *(int*)arg;
    char response[1024] = {0};
    while(clientRunning){
        if(!clientRunning){
            break;
        }
        memset(response, 0, sizeof(response));
        ssize_t len = recv(sock, response, sizeof(response), 0);
        if(len < 0){
            perror("recv");
            exit(EXIT_FAILURE);
        } else if(len == 0){
            break;
        }
        response[len] = '\0';
        printf("%s", response);
    }
    return NULL;
}
```

Each client sends its order details through a for loop inside main and each client has a handle_client thread that handles the responses coming from the server side and prints them.

```
pthread_t threads[numberOfClients];
client_sockets = (int*)malloc(sizeof(int) * numberOfClients);

printf("PID: %d\n", getpid());

for (int i = 0; i < numberOfClients; i++) {
    int *sock = (int*)malloc(sizeof(int));
    if ((*sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("Socket creation error\n");
        return -1;
    }

    client_sockets[i] = *sock;

    struct sockaddr_in serv_addr;
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);

    if (inet_pton(AF_INET, ip, &serv_addr.sin_addr) <= 0) {
        printf("Invalid address/ Address not supported\n");
        return -1;
    }

    if (connect(*sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection Failed");
        return -1;
    }

    int x = rand() % p;
    int y = rand() % q;
    send_order(*sock, i + 1, x, y, p, q);

    if(pthread_create(&threads[i], NULL, handle_client, sock) != 0){
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }

    sleep(1); // wait for the server to process the order
}
```

Main thread of client side sets up the order details and sends them through send_order function.

3. Test Results

- Type make to compile both PideShop and HungryVeryMuch
- Open up 2 terminals
- PideShop usage: ./PideShop [portNumber] [cookThreadSize] [deliverThreadSize] [deliverySpeed]
- HungryVeryMuch usage: ./HungryVeryMuch [ip] [portNumber] [totalOrders] [p] [q]
- Learn the ip address of server by entering ip a to the terminal

```
burak@DESKTOP-NK4AG86:/mnt/c/Users/User/Desktop/BilgiSAYAR/bilgisayar 6.yariyil/system prog/final$ . ./PideShop 8080 4 6 1
PideShop active waiting for connection...
100 new customers... Serving
Done serving client PID 2772
Thanks Cook 1 (29 orders) and Moto 1 (18 deliveries)
[]

Order 96 received
Order 96 prepared by cook 0
Order 96 is cooked by cook 0
Order 97 received
Order 97 prepared by cook 2
Order 97 is cooked by cook 2
Order 98 received
Order 98 prepared by cook 1
Order 98 is cooked by cook 1
Order 62 delivered by delivery 5
Order 99 received
Order 99 prepared by cook 1
Order 99 is cooked by cook 1
Order 71 delivered by delivery 1
Order 100 received
Order 100 prepared by cook 1
Order 100 is cooked by cook 1
Order 69 delivered by delivery 4
Order 67 delivered by delivery 0
Order 64 delivered by delivery 5
Order 72 delivered by delivery 1
Order 61 delivered by delivery 3
Order 70 delivered by delivery 4
Order 73 delivered by delivery 1
Order 74 delivered by delivery 0
Order 77 delivered by delivery 5
Order 55 delivered by delivery 2
Order 86 delivered by delivery 1
Order 89 delivered by delivery 2
Order 90 delivered by delivery 2
Order 80 delivered by delivery 3
Order 83 delivered by delivery 4
Order 78 delivered by delivery 5
Order 87 delivered by delivery 1
Order 91 delivered by delivery 2
Order 75 delivered by delivery 0
Order 79 delivered by delivery 5
Order 92 delivered by delivery 2
Order 95 delivered by delivery 5
Order 84 delivered by delivery 4
Order 88 delivered by delivery 1
Order 76 delivered by delivery 0
Order 93 delivered by delivery 2
Order 96 delivered by delivery 5
Order 81 delivered by delivery 3
Order 85 delivered by delivery 4
Order 98 delivered by delivery 1
Order 99 delivered by delivery 1
Order 94 delivered by delivery 2
Order 82 delivered by delivery 3
Order 100 delivered by delivery 1
Order 97 delivered by delivery 5
All customers served
burak@DESKTOP-NK4AG86:/mnt/c/Users/User/Desktop/BilgiSAYAR/bilgisayar 6.yariyil/system prog/final$ ./HungryVeryMuch 192.168.1.33 8080 100 10 20
```

I did not take every output of client side since it was so much, but I print every status of the orders at the client side.

```
burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$ ./PideShop 8080 4 6 1
PideShop active waiting for connection...
21 new customers... Serving
Done serving client PID 2980
Thanks Cook 0 (8 orders) and Moto 1 (5 deliveries)
18 new customers... Serving
Done serving client PID 2922
Thanks Cook 0 (12 orders) and Moto 1 (10 deliveries)
[]

Order 19 received
Order 19 prepared by cook 2
Order 19 is cooked by cook 2
Order 20 received
Order 20 prepared by cook 3
Order 20 is cooked by cook 3
Order 5 delivered by delivery 3
Order 21 received
Order 21 prepared by cook 0
Order 21 is cooked by cook 0
Order 4 delivered by delivery 2
Order 16 delivered by delivery 0
Order 9 delivered by delivery 4
Order 12 delivered by delivery 5
Order 6 delivered by delivery 3
Order 17 delivered by delivery 1
Order 13 delivered by delivery 5
Order 7 delivered by delivery 3
Order 10 delivered by delivery 4
Order 18 delivered by delivery 1
Order 19 delivered by delivery 1
Order 20 delivered by delivery 1
Order 21 delivered by delivery 1
All customers served
burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$ ./HungryVeryMuch 192.168.1.33 8080 18 10 20
PID: 2922
Order 1 received
Order 1 prepared by cook 1
Order 1 is cooked by cook 1
Order 2 received
Order 2 prepared by cook 2
Order 2 is cooked by cook 2
Order 3 received
Order 3 prepared by cook 1
Order 3 is cooked by cook 1
Order 4 received
Order 4 prepared by cook 0
Order 4 is cooked by cook 0
Order 5 received
Order 5 prepared by cook 1
Order 5 is cooked by cook 1
Order 1 delivered by delivery 2
Order 6 received
Order 6 prepared by cook 3
Order 6 is cooked by cook 3
Order 2 delivered by delivery 0
Order 7 received
Order 7 prepared by cook 1
Order 7 is cooked by cook 1
Order 3 delivered by delivery 0
Order 8 received
Order 8 prepared by cook 2
Order 8 is cooked by cook 2
Order 9 received
```

Handling two connections one after the other.

```
burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$ ./PideShop 8080 4 6 1
PideShop active waiting for connection...
18 new customers... Serving
^C.. Upps quitting... writing to log file
burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$

burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$ ./HungryVeryMuch 192.168.1.33 8080 18 10 20
PID: 3262
Order 1 received
Order 1 prepared by cook 0
Order 1 is cooked by cook 0
Connection Failed: Connection refused
burak@DESKTOP-NK4AG8G:/mnt/c/Users/User/Desktop/BİLGİSAYAR/bilgisayar 6.yarıyıl/system prog/final$
```

When SIGINT interrupt occurred at server side client side automatically closed since there does not exist a server socket anymore.

```
PideShop.c 4  pideShop.log x  HungryVeryMuch.c 4
pideShop.log
1 (2024-06-14 21:20:10) Order 1 prepared in 0.001002 second
2 Order 1 cooked
3 PideShop shutdown
4 Order 1 delivered to (8, 3) by delivery 0
5
```

Log file after server shutdown.

I did not test client side SIGINT because I could not manage to implement that.